

FastAPI vs Django vs Flask

Feature	FastAPI	Django	Flask
Definition	FastAPI is a modern, high-performance Python framework used mainly for building APIs quickly and efficiently.	Django is a high-level full-stack Python web framework used for rapid development of secure and scalable web applications.	Flask is a lightweight Python micro-framework used for developing simple and flexible web applications.
Framework Type	Modern high-performance Python API framework	Full-Stack Web Framework	Micro Web Framework
Main Goal	Fast API development	Complete web application development	Lightweight application development
Architecture	ASGI	MVT (Model View Template)	WSGI
Performance	Very High	Medium	High
Async Support	Full async support	Partial async support	Limited async support
Learning Difficulty	Medium	Medium to High	Easy
Built-in Features	API docs, validation	Admin, ORM, Auth, Templates	Minimal Features
ORM	External ORM	Built-in ORM	External ORM
Admin Panel	OAuth2 and JWT	Built-in authentication system	Extension-based
Template Engine	Jinja2	Django Templates	Jinja2
Routing System	Decorators	URLconf	Decorators
Form Handling	Manual handling	Built-in forms	External libraries

Validation	Automatic validation	Model/Form validation	Manual validation
Security	JWT, OAuth2	CSRF, middleware, authentication	Depends on extensions
Database Support	SQLAlchemy, PostgreSQL, MySQL	SQLite, PostgreSQL, MySQL, Oracle	SQLite, PostgreSQL, MySQL
Migrations	Alembic	Built-in migrations	Flask-Migrate
Testing Support	Pytest	Built-in testing tools	Pytest/Unittest
Scalability	Excellent	Excellent	Moderate
Documentation	Automatic Swagger/OpenAPI docs	Large official documentation	Simple documentation
Community Support	Growing community	Very large community	Large community
Flexibility	Highly flexible	Structured framework	Highly flexible
Development Speed	Very fast for APIs	Fast for complete applications	Fast for small apps
Best Use Case	APIs and microservices	Enterprise applications	Small projects and prototypes
Real-Time Support	WebSockets support	Limited	Extension-based
Static Files	Manual setup	Built-in management	Manual setup
API Development	Excellent	Good with DRF	Good with Flask-RESTful
Deployment	Uvicorn/Gunicorn	WSGI/ASGI servers	WSGI servers
Project Structure	Modular	Predefined structure	Flexible structure
Suitable For Beginners	Moderate	Good after basics	Best for beginners
Example Applications	AI APIs, backend services	E-commerce, CMS	Simple websites

1. FastAPI

FastAPI is a modern Python web framework used for building fast and efficient APIs. It is easy to learn, lightweight, and automatically generates API documentation.

FastAPI is a modern Python web framework, very efficient in building APIs. It is based on Python's type hints feature that has been added since Python 3.6 onwards. It is one of the fastest web frameworks of Python.

- ❖ As it works on the functionality of Starlette and Pydantic libraries, its performance is amongst the best and on par with that of NodeJS and Go.
- ❖ In addition to offering high performance, FastAPI offers significant speed for development, reduces human-induced errors in the code, is easy to learn and is completely production-ready.
- ❖ FastAPI is fully compatible with well-known standards of APIs, namely OpenAPI and JSON schema.

FastAPI has been developed by Sebastian Ramirez in Dec. 2018. FastAPI 0.68.0 is the currently available version.

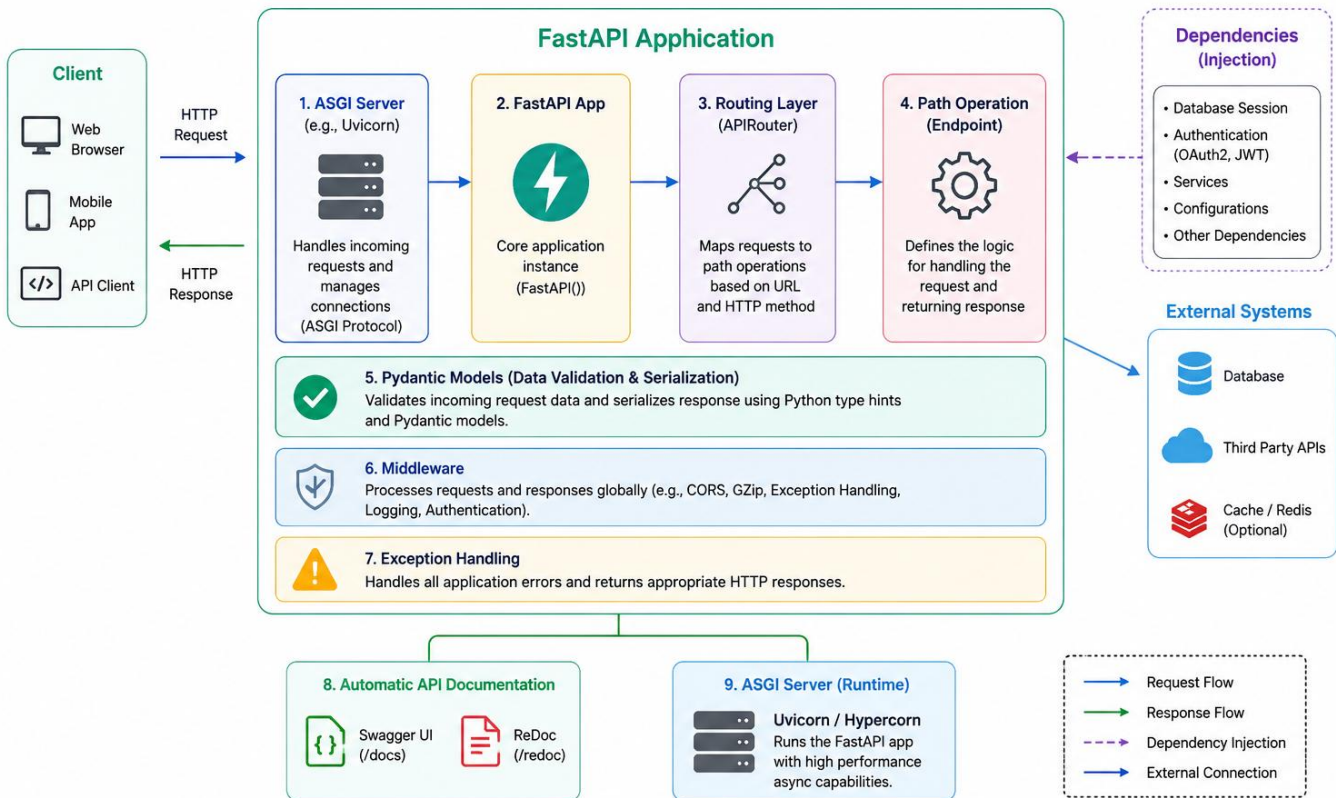
Features of FastAPI

- ✚ High speed performance
- ✚ Automatic Swagger documentation
- ✚ Async support
- ✚ Pydantic validation
- ✚ Easy API development
- ✚ Data Validation
- ✚ REST API development
- ✚ WebSocket support
- ✚ Dependency injection
- ✚ Type hint support

FastAPI Architecture

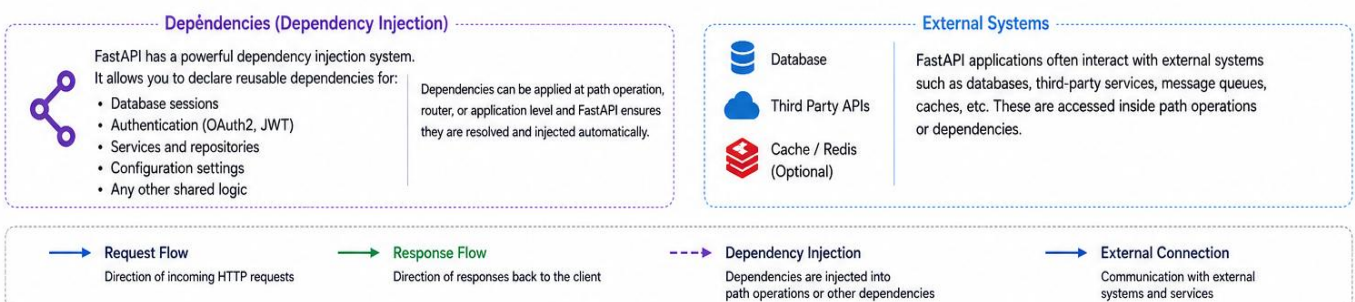
FastAPI follows ASGI(Asynchronous Server Gateway Interface) architecture.

FastAPI Architecture



FastAPI Architecture – Components with Description

#	Component	Description
1	 ASGI Server (e.g., Uvicorn)	Entry point of the application. Handles incoming HTTP requests using the ASGI (Asynchronous Server Gateway Interface) protocol. It manages network connections, concurrency, and forwards requests to the FastAPI application.
2	 FastAPI App (Core Application)	The central application instance created using `FastAPI()`. It holds the configuration, routes, middleware, exception handlers, and other settings. All parts of the application are registered here.
3	 Routing Layer (APIRouter)	Maps incoming requests to the appropriate path operations (endpoints) based on the URL path, HTTP method (GET, POST, PUT, DELETE, etc.) and other conditions. `APIRouter` helps in organizing routes in a modular way.
4	 Path Operation (Endpoint)	The actual function (path operation) that contains the business logic to handle the request. It receives validated data, interacts with dependencies, databases or services, and returns a response.
5	 Pydantic Models (Data Validation & Serialization)	Pydantic models are used to validate, parse and serialize request data and response data. They use Python type hints to ensure data correctness and automatically convert data to JSON for responses.
6	 Middleware	Functions that run before and after requests are processed. They can modify requests or responses globally. Examples: CORS, GZip Compression, Logging, Authentication, Error Tracking, etc.
7	 Exception Handling	Handles all errors and exceptions raised during request processing. FastAPI provides custom exception handlers to return appropriate HTTP status codes and messages.
8	 Automatic API Documentation	FastAPI automatically generates interactive API documentation based on your routes, parameters and models. Provides Swagger UI (at /docs) and ReDoc (at /redoc) for testing and exploring APIs.
9	 ASGI Server (Runtime) (e.g., Uvicorn / Hypercorn)	Runs the FastAPI application in an asynchronous environment with high performance. It supports features like hot reload, WebSockets, and background tasks.



Prerequisites

Before setting up FastAPI, make sure the following are installed on your system:

- Python
- pip
- Code Editor(VS Code recommended)

Download Python from the official website:

<https://www.python.org>

FastAPI – Environment Setup:

Step 1: Verify Python Installation

Open the terminal or command prompt and run:

```
python --version
```

Expected Output:

Python 3.x.x

Check pip installation:

```
pip --version
```

Step 2: Create a Project Folder

Create a new project directory:

```
mkdir fastapi_project cd fastapi_project
```

Step 3: Create Virtual Environment

Windows

```
python -m venv venv
```

Linux/macOS

```
python3 -m venv venv
```

Step 4: Activate Virtual Environment

Windows

```
venv\Scripts\activate
```

Linux/macOS

```
source venv/bin/activate
```

Step 5: Install FastAPI

Install FastAPI using pip:

```
pip install fastapi
```

Step 6: Install Uvicorn

Uvicorn is an ASGI server used to run FastAPI applications.

```
pip install uvicorn
```

You can also install both together:

```
pip install fastapi uvicorn
```

Step 7: Create First FastAPI Application

Create a file named main.py and add the following code:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def home():
    return {"message": "Hello FastAPI"}
```

Step 8: Run the FastAPI Server

Run the application using:

```
uvicorn main:app --reload
```

Explanation:

- main → Python filename
- app → FastAPI object
- --reload → Automatically reloads the server after changes

Step 9: Open in Browser

Open the following URL in your browser:

Eg: <http://127.0.0.1:8000>

Expected Output:

```
{"message": "Hello FastAPI"}
```

Step 10: API Documentation

FastAPI automatically provides API documentation.

Swagger URL

Eg: <http://127.0.0.1:8000/docs>

ReDoc

Eg: <http://127.0.0.1:8000/redoc>

FastAPI Routing:

Example:

```
@app.get("/users")
def get_users():
    return {"users": []}
```

Path Parameters:

Example:

```
@app.get("/items/{id}")
def get_item(id: int):
    return {"id": id}
```

Query Parameters:

Example:

```
@app.get("/search")
def search(q: str):
    return {"query": q}
```

Request Body Validation:

FastAPI uses Pydantic models.

Example:

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
```

Automatic API Documentation:

FastAPI automatically generates Swagger UI.

Swagger URL:

Eg: <http://127.0.0.1:8000/docs>

ReDoc

Eg: <http://127.0.0.1:8000/redoc>

FastAPI Database Support:

Popular ORM(Object Relational Mapping) options:

- SQLAlchemy
- Tortoise ORM

Install SQLAlchemy:

```
pip install sqlalchemy
```

FastAPI CRUD Example:

Create data:

```
@app.post("/users")
def create_user(user: User):
    return user
```

FastAPI Authentication:

Supports:

- JWT Authentication
- OAuth2

Example:

```
from fastapi.security import OAuth2PasswordBearer
```

FastAPI Middleware:

Example:

```
from fastapi.middleware.cors import CORSMiddleware
```

Used for

- CORS
- Logging
- Security

FastAPI Async Support:

Example :

```
@app.get("/")
async def home():
```

```
return {"message": "Async API"}
```

FastAPI WebSockets:

Example:

```
from fastapi import WebSocket
```

Supports real-time communication.

FastAPI Security:

Features:

- OAuth2
- JWT Tokens
- Password hashing
- CORS support

FastAPI Testing:

install pytest:

```
pip install pytest
```

Example:

```
from fastapi.testclient import Testclient
```

FastAPI Deployment:

Deploy using:

- Uvicorn
- Gunicorn
- Docker
- Nginx

Run production server:

```
Uvicorn main: app --host 0.0.0.0 --port 8000
```

Advantages of FastAPI:

- Extremely fast
- Automatic validation
- Automatic API docs
- Async support
- Easy API development

Disadvantages of FastAPI:

- Smaller ecosystem than Django
- Not ideal for full-stack applications
- Admin panel not included

Conclusion:

FastAPI environment setup is simple and efficient. By using virtual environments and Uvicorn, developers can build scalable and high-performance APIs quickly.

Django

Introduction to Django:

Django is a high-level Python web framework used for developing secure, scalable, and maintainable web applications quickly. It follows the MVT (Model View Template) architecture and provides many built-in features like ORM, admin panel, authentication system, forms, migrations, and security tools.

Django was designed to simplify common web development tasks and encourage rapid development with clean and reusable code.

Features of Django:

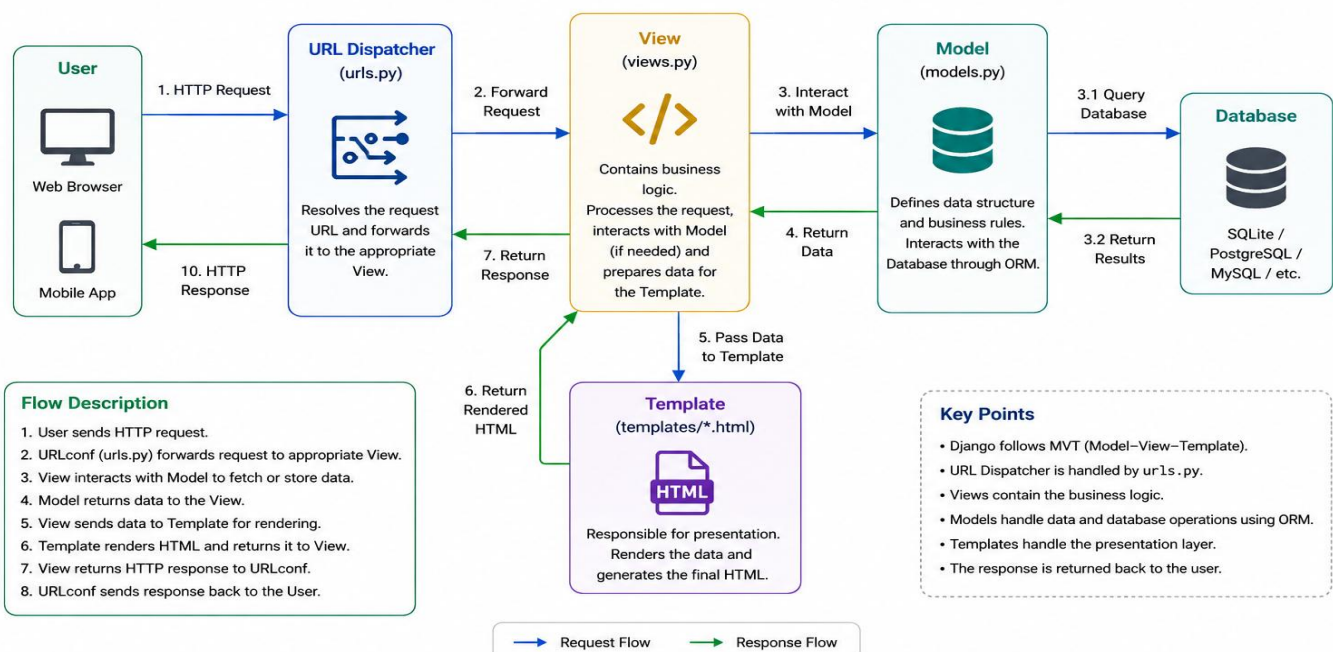
- Fast development
- Built-in admin panel
- ORM (Object Relational Mapper)
- URL routing system
- Authentication system
- Template engine
- Form handling
- Security protection
- Database migrations
- Testing framework
- Static file management
- Internationalization support

Django Architecture (MVT):

Django follows the MVT architecture.

Django Architecture (MVT)

Django follows the MVT (Model – View – Template) architectural pattern.
It emphasizes reusability and separation of concerns.



Django Architecture Components

	1. User / Client	End user who interacts with the Django application through a web browser, mobile app or any API client by sending HTTP requests.
	2. URL Dispatcher (urls.py)	Receives incoming HTTP requests and matches the requested URL with defined URL patterns. It then forwards the request to the appropriate view.
	3. View (views.py)	Contains the business logic of the application. It processes the request, validates data, interacts with models (if required) and prepares data for templates or returns a response.
	4. Model (models.py)	Defines the structure of data and business rules. It interacts with the database using Django ORM to perform CRUD operations.
	5. Database	Stores all application data permanently. Django supports multiple databases such as SQLite, PostgreSQL, MySQL, Oracle, MariaDB, etc.
	6. Template (templates/*.html)	Handles the presentation layer. Receives data from views and renders dynamic HTML using Django template engine, filters and template tags.
	7. HTTP Response	The final response generated by the view (rendered HTML, JSON, XML, file, etc.) is sent back to the user through URL Dispatcher.
	8. Middleware	A layer of hooks executed before the request reaches the view and after the response is returned. Handles tasks like authentication, sessions, CSRF protection, common security and request/response processing.
	9. Admin Panel	Built-in admin interface provided by Django to manage application data. It is automatically generated from models with minimal configuration.
	10. Forms Framework	Provides tools to create, validate and process HTML forms for user input. It handles form rendering, validation and error messages easily.
	11. Authentication System	Provides built-in authentication features such as user registration, login, logout, password hashing, permissions, groups and user management.
	12. Static Files	Manages static resources like CSS, JavaScript, images and fonts. These files are served to the client as they are without any processing.
	13. Template Engine	Django's template engine processes templates with variables, tags, filters and inheritance to generate dynamic content.
	14. Security Layer	Django provides built-in security features to protect against common web vulnerabilities like CSRF, XSS, SQL Injection, Clickjacking, session hijacking and more.
	15. HTTP Response (Final Output)	The processed output generated by Django is sent back to the user as the final HTTP response.

Flow of Django Request:

1. User sends request
2. URL dispatcher receives request
3. View processes request
4. Model interacts with database
5. Template renders response
6. Response sent back to user

Installing Django

Step 1: Install Python

Download Python from the official website.

```
python --version
```

Expected Output:

Python 3.x.x

Step 2: Install pip

pip is Python's package manager.

```
pip --version
```

Step 3: Create Virtual Environment

Windows

```
python -m venv env
```

Linux/macOS

```
source env/bin/activate
```

Step 4: Activate Virtual Environment

Windows

```
venv\Scripts\activate
```

Linux/Mac

```
source env/bin/activate
```

Step 5: Install Django

```
pip install django
```

Verify installation:

```
python -m django --version
```

Creating a Django Project:

Create project:

```
django-admin startproject mysite
```

Move into project directory:

```
cd mysite
```

Run development server:

```
python manage.py runserver
```

Server starts at:

```
http://127.0.0.1:8000/
```

Django Project Structure

mysite/

```
manage.py
```

```
mysite/
```

```
__init__.py
```

```
settings.py
```

```
urls.py
```

```
wsgi.py
```

Explanation:

manage.py

Command-line utility for interacting with Django project.

settings.py

Contains project settings and configurations.

urls.py

Handles URL routing.

wsgi.py

Used for deployment.

Creating Django App

Create app:

```
python manage.py startapp polls
```

Generated structure:

polls/

admin.py

apps.py

models.py

tests.py

views.py

Django Models

Models define database structure.

Example:

```
from django.db import models
```

```
class Question(models.Model):
```

```
    question_text = models.CharField(max_length=200)
```

```
    pub_date = models.DateTimeField()
```

Common Field Types

Field	Purpose
CharField	String data
TextField	Large text
IntegerField	Integer values
DateField	Date
DateTimeField	Date and time
BooleanField	True/False

Database Migrations:

Create Migration

```
python manage.py makemigrations
```

Apply Migration

```
python manage.py migrate
```

SQL Preview

```
python manage.py sqlmigrate app_name 0001
```

Django ORM:

Django ORM allows database interaction using Python code.

Create Object

```
Question.objects.create(question_text="Python")
```

Retrieve Data

```
Question.objects.all()
```

Filter Data

```
Question.objects.filter(id=1)
```

Update Data

```
obj.question_text = "Django"
```

```
obj.save()
```

Delete Data

```
obj.delete()
```

Django Views:

Views contain business logic.

Example:

```
from django.http import HttpResponse
def index(request):
    return HttpResponse("Hello Django")
```

URL Routing:

URLs map requests to views.

Example:

```
from django.urls import path
from . import views
urlpatterns = [
    path("", views.index),
]
```

Django Templates:

Templates handle frontend UI.

Example:

```
<h1>{{ title }}</h1>
```

Template Features:

- Variables
- Loops
- Conditions
- Template inheritance
- Filters

Template Inheritance:

Base Template:

```
{% block content %}
```

```
{% endblock %}
```

Child Template:

```
{% extends 'base.html' %}
```

Django Admin Panel:

Django provides automatic admin interface.

Create Superuser

```
python manage.py createsuperuser
```

Register Model

```
from django.contrib import admin
from .models import Question
admin.site.register(Question)
```

Admin URL:

`http://127.0.0.1:8000/admin/`

Forms in Django

Django provides form handling framework.

Example:

```
from django import forms
class ContactForm(forms.Form):
    name = forms.CharField()
```

Authentication System:

Django includes built-in authentication.

Features

- Login
- Logout
- Password hashing
- Permissions
- User management

Security Features:

Django provides protection against:

- CSRF attacks
- Clickjacking
- SQL Injection
- XSS attacks

Security middleware is built-in.

Static Files:

Static files include:

- CSS
- JavaScript
- Images

Configuration:

```
STATIC_URL = '/static/'
```

Django Testing:

Run tests:

```
python manage.py test
```

Django includes built-in testing framework.

Django Middleware:

Middleware processes requests and responses globally.

Examples:

- Authentication middleware
- Session middleware
- Security middleware

Django Signals:

Signals allow communication between applications.

Example:

```
from django.db.models.signals import post_save
```

Caching Framework:

Django supports caching for performance optimization.

Supported backends:

- Memcached
- Database cache
- File-based cache

Internationalization:

Django supports multiple languages and localization.

Features:

- Translation
- Time zones
- Localized formatting

Deployment:

Common deployment servers:

- Gunicorn
- uWSGI
- Apache
- Nginx

Advantages of Django:

- Rapid development
- Secure framework
- Scalable
- Reusable components
- Excellent documentation

Disadvantages of Django:

- Heavy for small projects
- Large framework
- More configuration required

Conclusion:

Django is one of the most powerful Python web frameworks used for building secure and scalable web applications. It provides many built-in tools such as ORM, admin panel, authentication, templates, migrations, and security features which simplify web development.

Flask

Introduction to Flask:

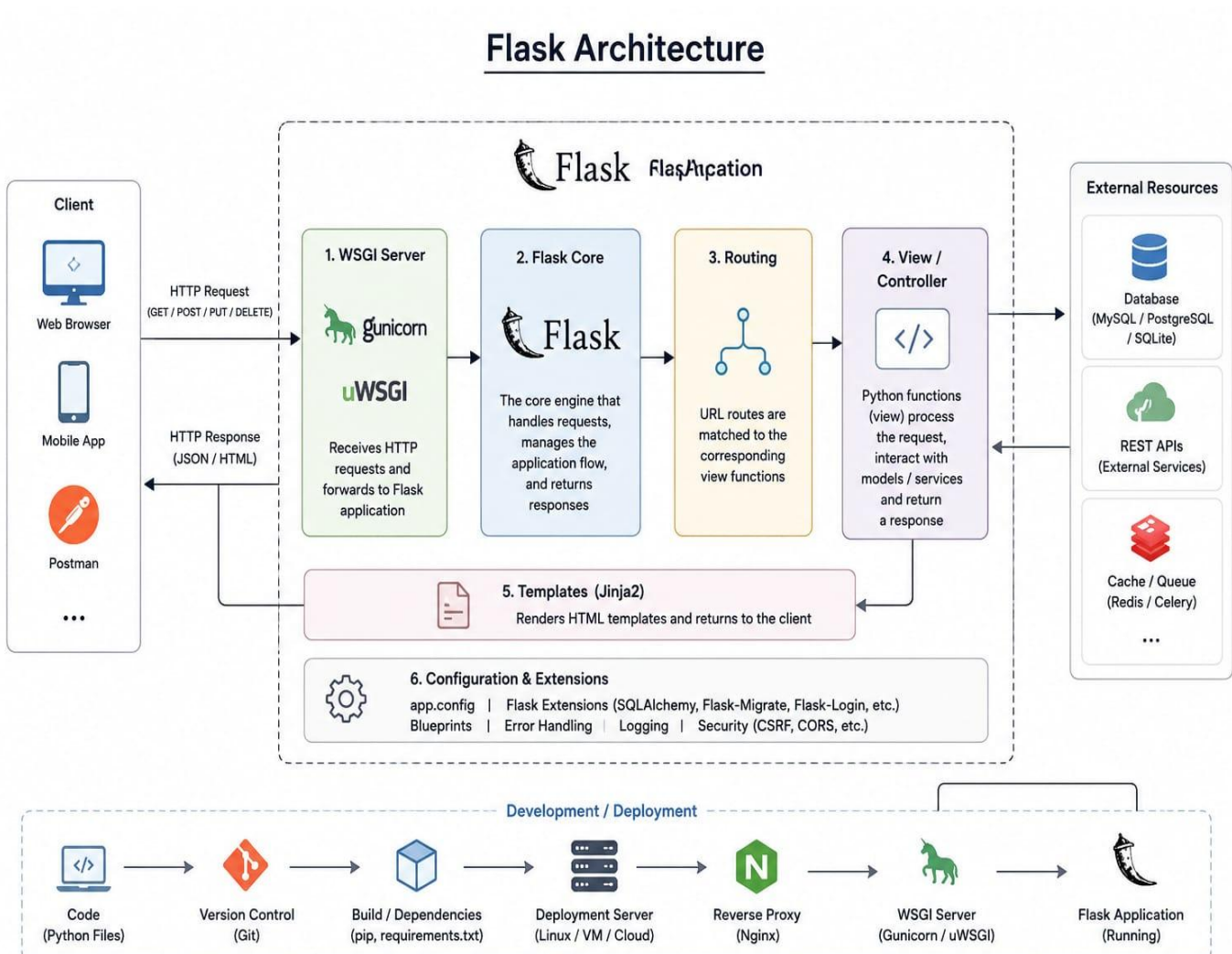
Flask is a lightweight Python web framework used for building web applications and APIs. It is called a micro-framework because it provides only the essential tools needed for web development while allowing developers to choose additional libraries as required. Flask is simple, flexible, and beginner-friendly.

Features of Flask:

- Lightweight framework
- Simple routing system
- Jinja2 template engine
- REST API support
- Flexible architecture
- Easy to learn
- Extension support
- Debugging support

Flask Architecture:

Flask follows a simple WSGI-based architecture.



Flask Architecture – Components with Description

No.	Component	Description
1	 Client (Browser / API Client)	The end user interacts with the Flask application using a web browser, mobile app or any API client by sending HTTP requests.
2	 URL Routing	Flask uses the routing system defined by the <code>@app.route()</code> decorator to map incoming URLs to the corresponding view functions.
3	 View Function (Route Handler)	View functions contain the business logic of the application. They process the request, interact with models or other services, and return a response.
4	 Models (Data / ORM)	Models represent the data structure and interact with the database. Flask does not include a built-in ORM but supports extensions like SQLAlchemy.
5	 Database	The database stores application data permanently. Flask supports any database through extensions like SQLAlchemy (SQLite, MySQL, PostgreSQL, etc.).
6	 Templates (Jinja2)	Flask uses the Jinja2 template engine to render HTML pages. Templates receive data from views and generate dynamic content.
7	 Static Files	Static files include CSS, JavaScript, images and other assets. They are served to the client as they are without any processing.
8	 Forms (Flask-WTF)	Flask-WTF extension provides form handling, validation, CSRF protection and easy processing of user input.
9	 Session Management	Flask provides secure session management using signed cookies on the client side or server-side sessions using extensions like Flask-Session.
10	 Authentication (Flask-Login)	Handles user authentication related tasks such as login, logout, user session management and protected routes.
11	 Extensions	Flask is extensible. Common extensions include SQLAlchemy (ORM), Flask-Migrate (migrations), Flask-Login (auth), Flask-WTF (forms), Flask-RESTful (APIs) and more.
12	 Middleware / Hooks	Flask supports request and response processing using middleware, <code>before_request</code> , <code>after_request</code> , <code>teardown_request</code> and other hooks.
13	 Error Handling	Handles exceptions and returns appropriate error responses using custom error handlers and HTTP status codes.
14	 APIs (JSON Response)	Flask is lightweight and ideal for building RESTful APIs. It can return JSON, XML or any other format based on requirements.
15	 WSGI Server	Flask application runs on a WSGI server (like Werkzeug in development, Gunicorn, uWSGI in production) which handles requests and serves the application.
16	 HTTP Response	The final response generated by the view (HTML, JSON, file, etc.) is returned to the client through the WSGI server.

Installing Flask

Step 1: Install Python

```
python --version
```

Step 2: Create Virtual Environment

```
python -m venv env
```

Activate environment:

Windows

```
env\Scripts\activate
```

Linux/Mac

```
source env/bin/activate
```

Step 3: Install Flask

```
pip install flask
```

Verify installation:

```
pip show flask
```

Creating Flask Application:

Create file:

```
app.py
```

Example:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')

```

```
def home():
    return "Hello Flask"
if __name__ == '__main__':
    app.run(debug=True)Run application:
python app.py
```

Flask Project Structure:

```
project/
  app.py
  templates/
  static/
```

Flask Routing:

Routing maps URLs to functions.

Example:

```
@app.route('/about')
def about():
    return "About Page"
```

Flask Templates:

Flask uses Jinja2 template engine.

Example:

```
<h1>{{ title }}</h1>
```

Render template:

```
from flask import render_template
@app.route('/')
def home():
    return render_template('index.html')
```

Template Inheritance:

Base Template:

```
{% block content %}
{% endblock %}
```

Child Template:

```
{% extends 'base.html' %}
```

Flask Static Files:

Static files include:

- CSS
- JavaScript
- Images

Folder structure:

```
static/
```

Flask Forms:

Flask uses WTForms extension.

Install:

```
pip install flask-wtf
```

Example:

```
from flask_wtf import FlaskForm
```

Flask Database Support:

Flask supports databases using extensions.

Popular ORM:

- SQLAlchemy

Install:

```
pip install flask-sqlalchemy
```

Example:

```
from flask_sqlalchemy import SQLAlchemy
```

Flask Authentication:

Authentication handled using extensions.

Popular extension:

```
pip install flask-login
```

Features:

- Login
- Logout
- Session management

Flask REST API:

Install Flask RESTful:

```
pip install flask-restful
```

Example API:

```
from flask import jsonify
```

```
@app.route('/api')
```

```
def api():
```

```
    return jsonify({"message": "Hello API"})
```

Flask Middleware:

Flask supports middleware through WSGI middleware.

Examples:

- Logging middleware
- Authentication middleware

Flask Security:

Security features depend on extensions.

Protection includes:

- CSRF protection
- Session security
- Password hashing

Flask Testing:

Install pytest:

pip install pytest

Run tests:

Pytest

Flask Deployment:

Popular deployment servers:

- Gunicorn
- uWSGI
- Nginx

Run with Gunicorn:

gunicorn app:app

Advantages of Flask:

- Simple and lightweight
- Flexible framework
- Easy for beginners
- Easy API development

Disadvantages of Flask:

- No built-in admin panel
- Requires external libraries
- Manual configuration for large projects

Conclusion:

Flask is a lightweight and flexible Python framework suitable for small to medium applications and APIs. It is beginner-friendly and allows developers to add only the required components.

Best suited for:

- Small applications
- APIs
- Prototypes
- Learning web development